Version Control Systems

Student Workbook 1b – Git and GitHub

Version 7.1 Y

Table of Contents

Module 1 Version Control System and Git....3
Section 1–1 Git 4
What is Version Control? 5
Version Control Systems 6
Section 1–2 Git Basics and the Local Repository 7
Git 9
CLI vs IDE vs GUI 10
Managing our Projects using the IntelliJ IDE 13
Section 1–3 GitHub 15
GitHub - A Remote Repository Service 16
GitHub User Interface 17
Creating a Remote Repository 18
Exercise – My First Repo 19
Creating our First Local Repository 20
Exercise – Project Upload 25

Module 1

Version Control System and Git

Section 1–1

Git

What is Version Control?

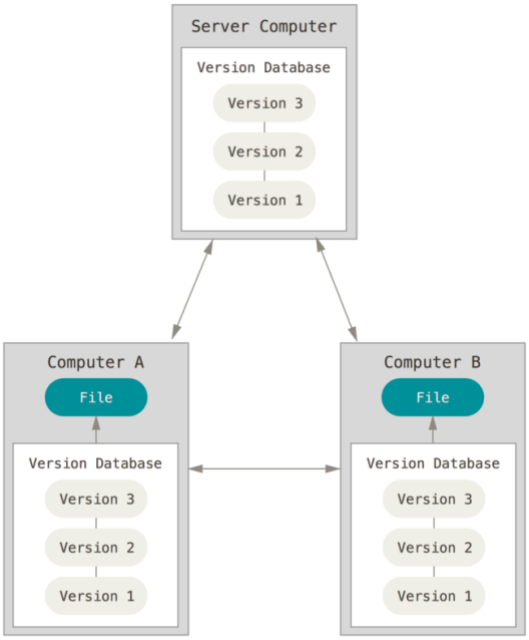
- **Version control systems help developers keep track of changes to their source code over time**
  - Commonly referred to as SCM (***Source Code Management***)
  - Or VCS (***Version Control Systems***)
- **The idea is to keep track of every modification made to the code base**
- **You "commit" your changes and store them in a repository**
  - Each time you add "something that works" you generally do a commit
  - This might be a several times per hour if you are productive
- **VCS features allow you to roll back to a previous commit if needed**
  - You can also "roll forward" if you've already rolled back
- **Tools also let you compare code in different commits**
  - It can help you figure out what changes might have caused a bug
- **VCS allows teams of developers to collaborate on one code base**
- **Version control systems can be:**
  - centralized
  - distributed

Version Control Systems

- **Distributed Version Control systems like Git do *not* rely on a central server to store all the versions of a project's files**
  - Each developer has a copy of the repository and with all versions

- **Cloud based services like GitHub, GitLab and BitBucket can provide a centralized (additional) copy of a repository**
  - To work on a project, each developer could create a repository on their local machine or "clone" an existing repository from the cloud service

- **The developer's local repository would contain the full history of the project**
  - A developer can make changes to the code and commit those changes to their local repository
  - When they are ready to share changes with other developers, they "push" their changes and all the attached history to the remote repository
  - Once pushed, other developers can now update their local copies with those changes.



- **Typical Workflow for Git**
  - Developer checks out code from version control or pulls down the latest changes to code already checked out
  - Developer makes some changes and tests their work
  - Developer makes a local "commit" that represents the changes made
  - Developer repeats this cycle locally until they are ready to share their changes with other developers
  - Developer commits changes back to the central repository for others to "pull" into their local copies or provide a patch file to another developer if they choose not to use a service like Github

Section 1–2

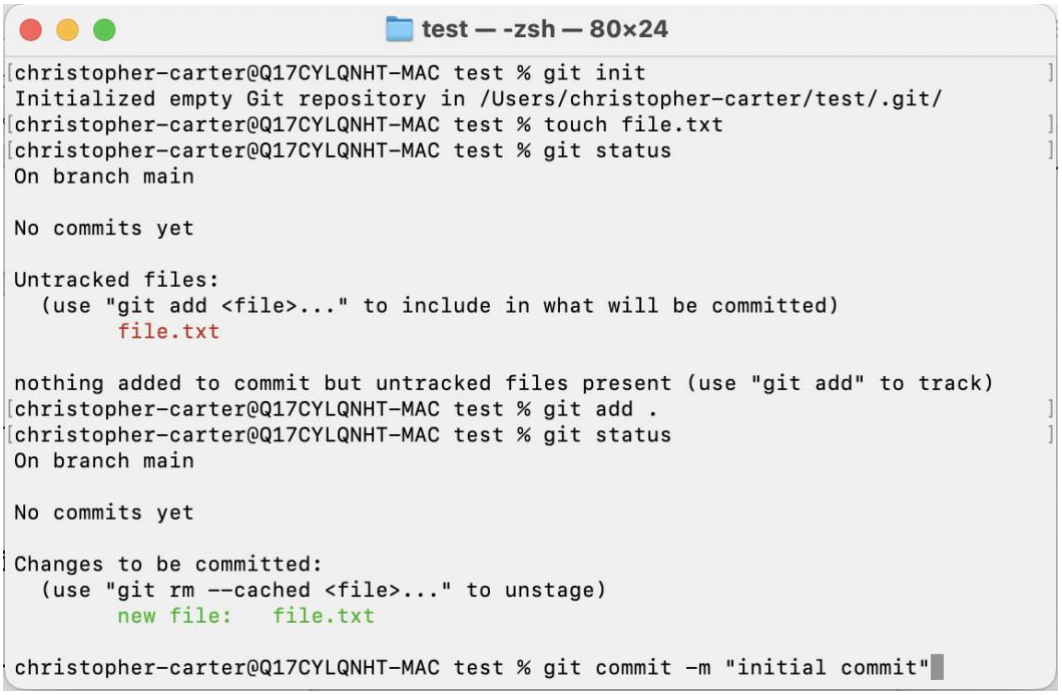
Git Basics and the Local Repository

Git

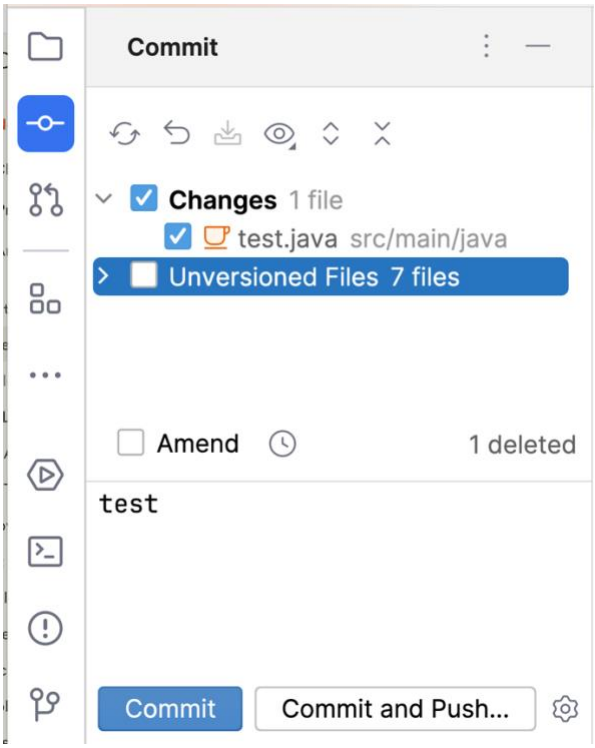
- **Git is a free, open source distributed version control system**
  - It can handle both small and very large projects
- **It is easy to learn and has a tiny footprint**
  - Most importantly, it is very fast
- **Before you start using Git, install it on your developer machine from `https://git-scm.com/downloads`**
  - Note: Your machine should have been configured before the class started and Git will already be on it
- **To run Git commands, you can use:**
  - Git Bash shell, terminal, a bash shell,
  - or the Windows command prompt window
- **NOTE: We will use IntelliJ's git tooling**
  - Afterwards, you can use Git Bash or the Command Prompt window. Choose whichever you prefer or your company recommends

CLI vs IDE vs GUI

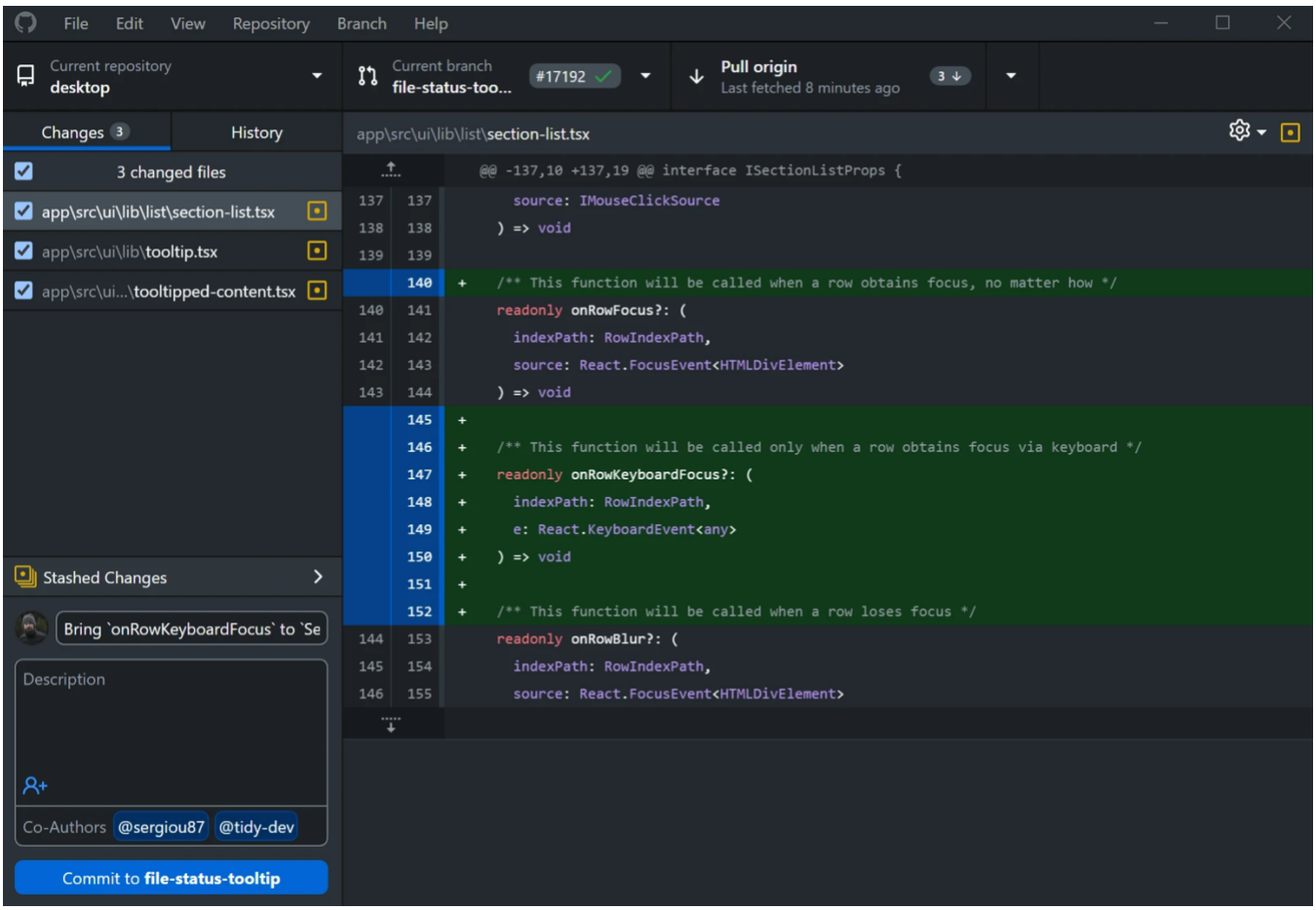
- **Command Line Interface (CLI)**
  - Direct interaction with git through terminal commands like `git add`, `git commit`, `git push`
  - Provides complete access to all git features and advanced functionality
  - Requires memorizing commands but offers precise control and is consistent across all operating systems
  - Essential for automation, scripting, and troubleshooting complex git issues



- **Integrated Development Environment (IDE)**
  - Git functionality built directly into code editors like VS Code, IntelliJ, or Visual Studio
  - Seamlessly integrates version control with your coding workflow
  - Offers visual indicators for file changes, inline blame annotations, and commit history within your editor
  - Combines the convenience of GUI tools with the context of your actual code



- **Graphical User Interface (GUI)**
  - Standalone visual applications like GitHub Desktop, GitKraken, or Sourcetree
  - Uses buttons, menus, and visual representations instead of text commands
  - Makes complex operations like merge conflict resolution and branch visualization more intuitive
  - Great for beginners or visual learners who prefer point-and-click interactions over command memorization



Managing our Projects using the IntelliJ IDE

- **IntelliJ provides built-in git support with visual indicators showing file status changes (by default it's green for new, blue for modified, red for conflicts)**

- **We can access git operations through the VCS menu, right-click context menus, or the Git tool window at the left of the screen**

- **We can view commit history, create branches, and merge changes directly within our development environment**

Section 1–3

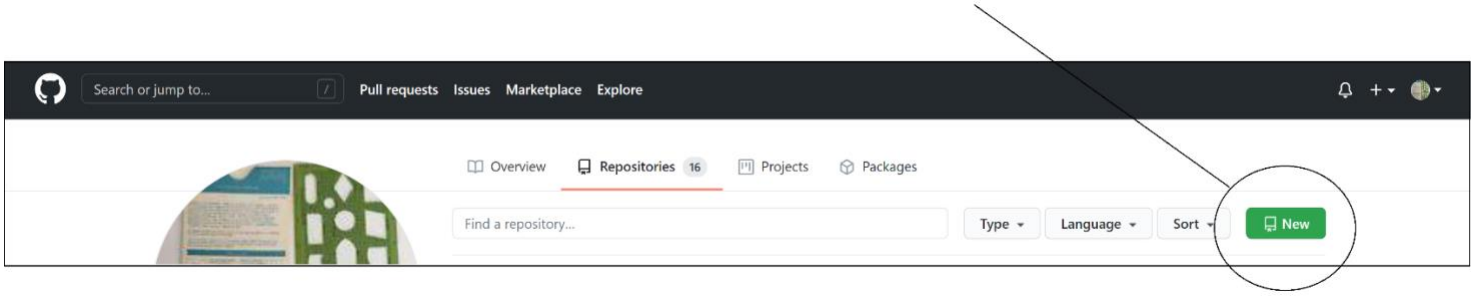
GitHub

GitHub - A Remote Repository Service

- **GitHub is a cloud-based service that can be used to store and manage remote git repositories**
  - Think of it as a backup for your local repo (!)
- **In addition to providing cloud storage for your repositories, it lets you make your Git repositories available to other developers**
- **Anyone can sign up at GitHub and host public code repositories for free**
  - This makes GitHub very popular as a hosting site for open-source projects
  - Sign up for an account at: github.com
- **GitHub is a for-profit company and makes money by selling hosted private code repositories and other team-based development services**
- **Many organizations host their own internal GitHub cloud service so that they have complete control over its visibility**
- **However, we will use the public GitHub in this class**

GitHub User Interface

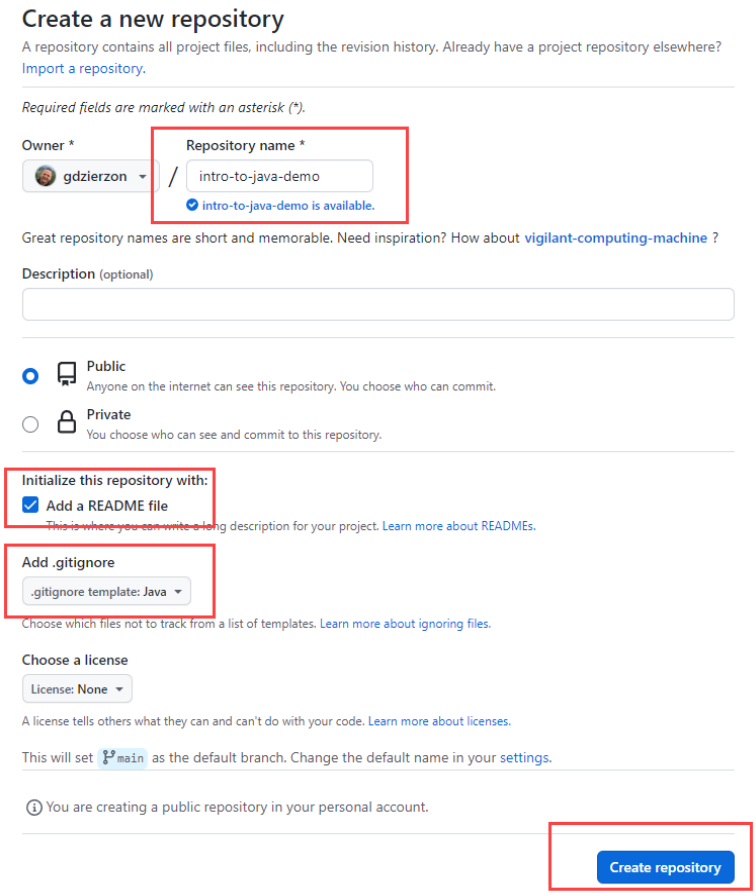
- **GitHub has a web-based graphical interface**
  - It allows you to create new repositories easily



- **It has opinions on how to grant access to your code as well as how people can contribute to your code**

- **It provides several collaboration features for your project, such as:**
  - basic repo management
  - wikis

Creating a Remote Repository



- **After clicking the New button, you can:**
  - Name the repo
  - Set the visibility of the repo (public / private)
  - Accept the default main branch name as 'main' or change it to something else
  - Until recently, it defaulted to 'master'
- **There may be small differences on your internal GitHub**

Exercise – My First Repo

In this exercise, you will create a GitHub repository

EXERCISE 1

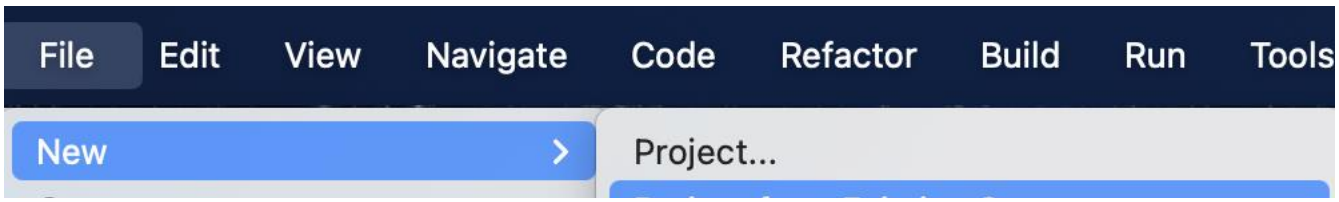
Step 1: In your browser navigate to `https://github.com` and create a new repository with the following settings:

- Name: `my-first-repo`
- Add a description
- Have no other selections made

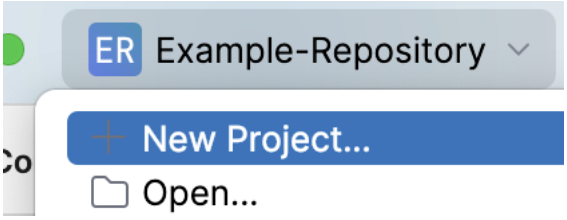
Step 2: Send a link to your GitHub repo to your instructor

Creating our First Local Repository

- **We will start by creating a new project using IntelliJ**
  - For this project, we won't create any files or write any code!
- **Either select the new Project, under the File dropdown menu**



- **Or select the project you are on, and click 'New Project…"**



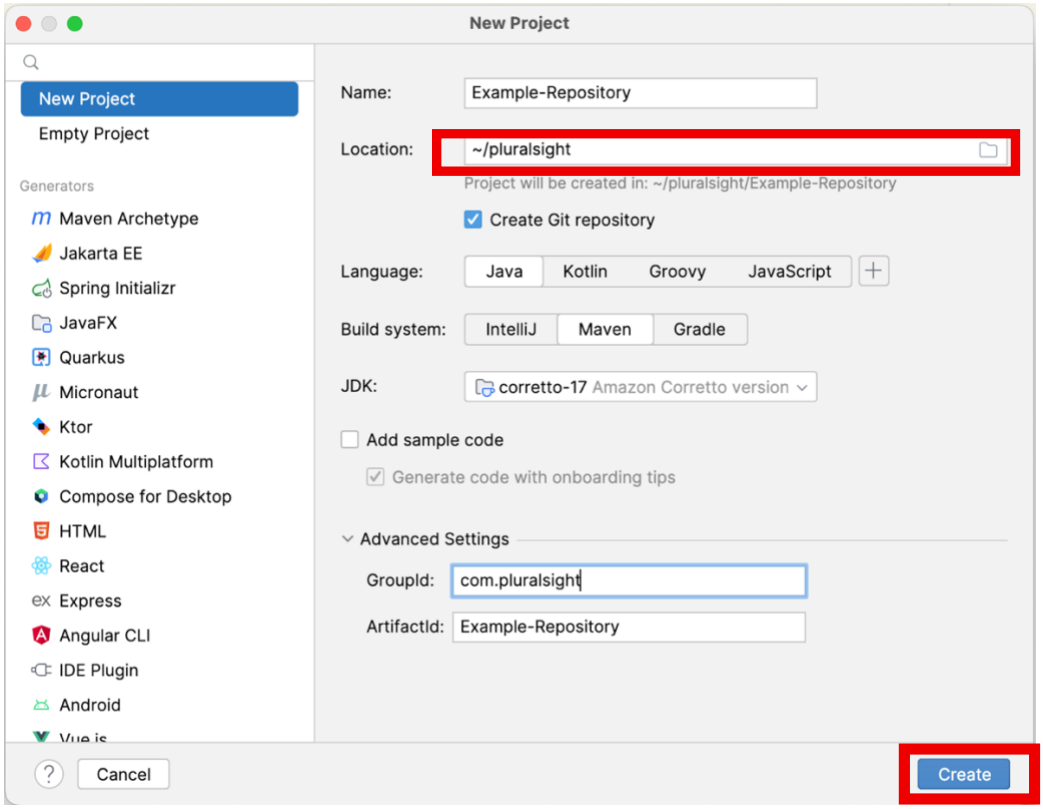
- **Within the New Project Menu, ensure you have a name, location, and 'Create Git repository' selected**
- **The Location should be:**
  - Mac/Linux: `~/pluralsight`
  - Windows: `C:/Users/[username]/pluralsight`

- **Create Git repository: selected**

- **The Language: `Java`**

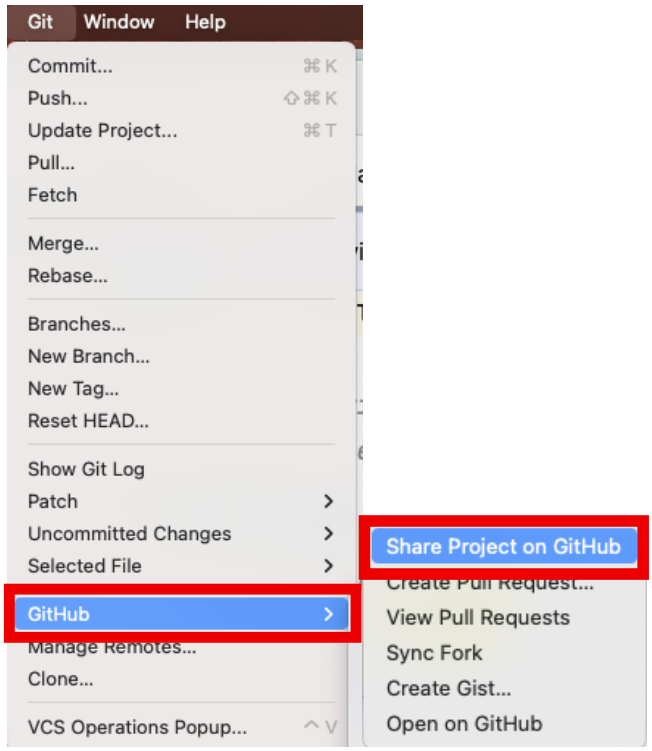
- **Build System: `Maven`**

- **JDK: `corretto-17 Amazon Corretto`**



- **On the top navigation bar, select the git dropdown**

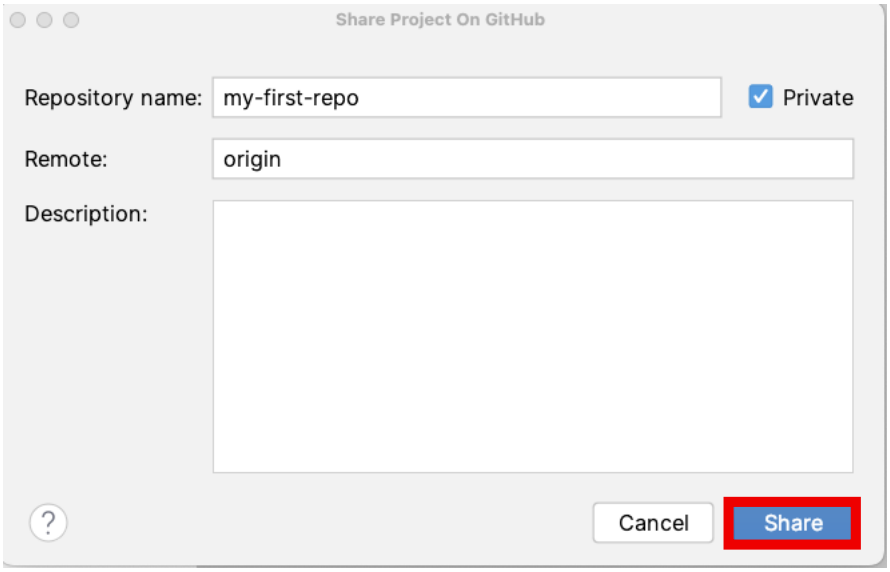
- **Go to 'GitHub' and select 'Share Project on GitHub'**



- **If this is your first time sharing a project to GitHub, you will be asked to connect your account to IntelliJ. Follow the steps presented to you**

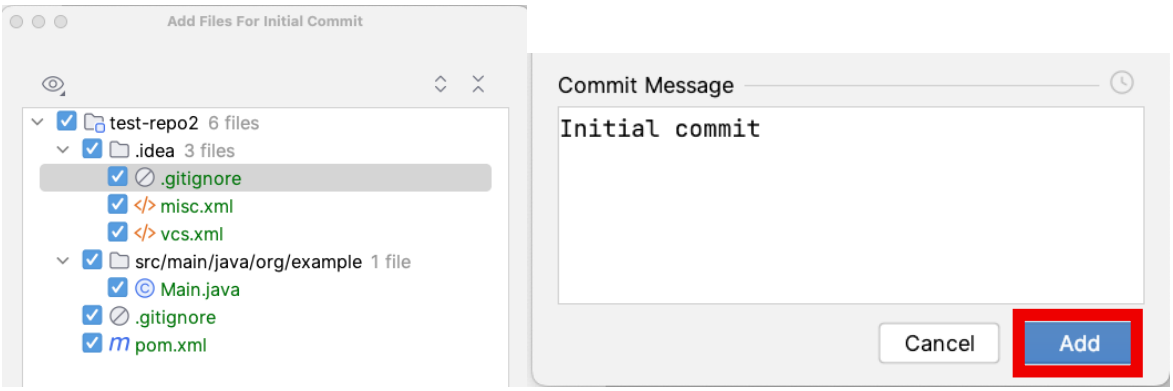
- **The next pop-up should allow you to Share your repository to GitHub**

- **Click 'Share'**



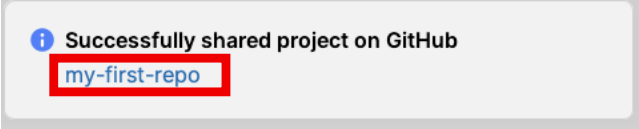
- **You will be asked to select the files for an Initial Commit**

- **Select all files for now, and type 'Initial Commit' as a message for your first commit! Click 'Add'!**



- **After you've clicked 'Add' you should see an alert in the bottom left of your IntelliJ application**

- **Click the link with the name of your project!**



- **After you've opened that link, you should see your repository on Github.com!**

Exercise – Project Upload

In this exercise you will connect a local project to github.com

- **Step 1:** Start a new project in IntelliJ

  Within the New Project Menu, ensure you have a name, location, and '**Create Git repository'** selected

  The Name: **`my-second-repo`**

  The Location should be:

  Mac/Linux: `~/pluralsight`

  Windows: `C:/Users/[username]/pluralsight`

  The Language: **Java**

  Build System: **Maven**

  JDK: **corretto-17 Amazon Corretto**

Step 3: Follow the previous steps of sharing your project to Github

Step 4: Open the repository on github!

Step 5: Send your Instructor a message letting them know you've updated the repository with your first commit